

コマ 7: 再帰的な変数宣言収集

今日のゴール

関数本体の深い部分にある変数宣言も事前に収集できるようにする。

コマ 4 では `_reset_func_state()` で関数直下の `Decl` だけを `collect_decls` して `frame_size` を計算していた。しかし、次のように `if` の中に宣言があると、その宣言を見落としてしまう。

```
int main() {
    int x;
    x = 3;
    if (x == 3) {
        int y;      // ← この宣言をコマ 4 では見落とす
        y = 4;
        return x + y;
    }
    return 0;
}
```

コマ 7 では `collect_decls()` を再帰的に拡張し、`Block / If / While / For` の内側まで宣言を収集できるようにする。

1 この回で扱う範囲

この回では新しい C の構文は導入しない。これまでに扱ったすべての機能を対象とし、その上で宣言収集の範囲を拡大する。

改良点	内容
再帰的宣言収集	<code>Block, If, While, For</code> の中の <code>Decl</code> も <code>collect_decls</code> で収集する

関数呼び出しや再帰は第 08 回で扱う。

2 コマ 4 の制限

コマ 4 の `collect_decls()` は関数直下の `Decl` だけを収集していた。

```
# コマ 4 の実装例
def collect_decls(self, node: Node) -> None:
    match node.kind:
        case 'Decl':
            self.collect_decls_Decl(node)
        case _:
            pass
```

これでは、`if` や `while` の中の `int y;` を確保できず、実行時に正しい位置に変数領域が確保されない。

3 collect_decls を拡張する

`collect_decls()` を `match node.kind` で拡張し、制御構文の中も再帰的に調べるようにする。

node.kind	処理
'Decl'	<code>self.collect_decls_Decl(node)</code> — 変数を登録
'Block'	<code>self.collect_decls_Block(node)</code> — <code>stmts</code> を再帰的に収集
'If'	<code>self.collect_decls_If(node)</code> — <code>then</code> と <code>else_</code> の中を 収集
'While'	<code>self.collect_decls_While(node)</code> — <code>body</code> の中を収集
'For'	<code>self.collect_decls_For(node)</code> — <code>body</code> の中を収集

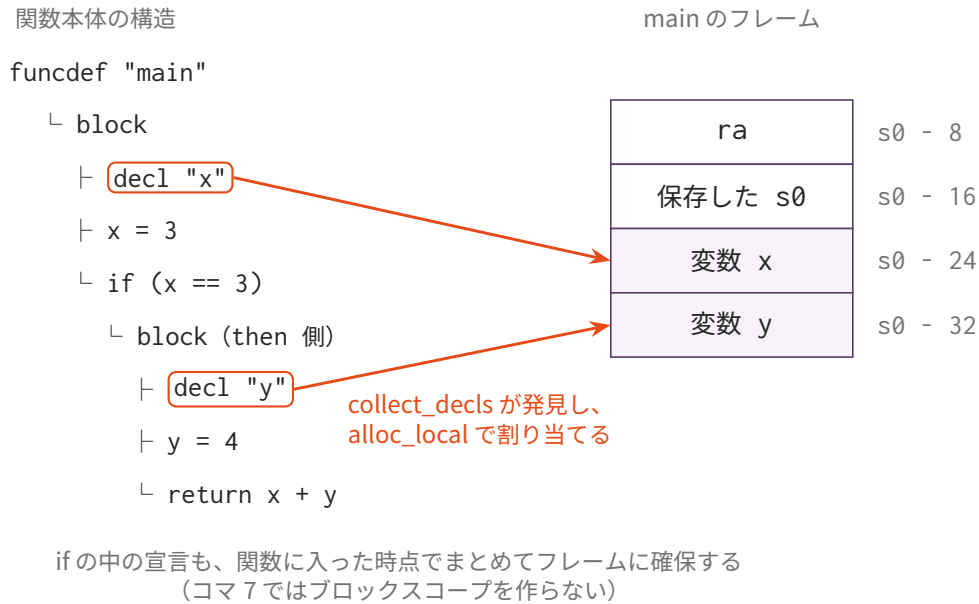


図 1 collect_decls の再帰走査と、フレームへの一括割り当て

コマ 7 では、C の正確なブロックスコープはまだ実装しない。関数内に出てくるすべてのローカル変数を、関数全体のフレームに一律で確保する。

if の中の decl "y" も走査で発見し、x と同じように関数フレームへ割り当てる。

同じ関数内で同じ名前の変数を 2 回宣言するケースは扱わない。

4 _reset_func_state の更新

コマ 6 まででは _reset_func_state() の処理が分散していた（コマ 4 の TODO とコマ 6 のラベルスタック初期化が別々に行われていた）。コマ 7 では _reset_func_state() で次の初期化をひとつにまとめる。

1. self._locals をクリアする
2. self._stack_offset を 0 にする
3. self._break_stack / self._continue_stack をクリアする
4. self._ret_label を self._new_label() で生成する
5. self.collect_decls(node.body) で再帰的に宣言を収集する
6. align_to(self._stack_offset, 16) を frame_size として返す

これまで main() の中で行っていたラベルスタック等のクリア処理も、_reset_func_state

に統合する。

5 `_emit_func_body` の変更

コマ6までは `_emit_func_body` で `body.stmts` を1つずつ `gen_stmt` していた。しかし `If` の `else_` に `Block` が含まれる場合など、文構造を保ったままの処理が必要になる。

そのため、`_emit_func_body` では `body` 全体を `gen_stmt(node)` に渡す形にする。

```
def _emit_func_body(self, body: Node) -> None:
    self.gen_stmt(body) # Block ノードとして扱う
```

6 編集するファイル

- `mycc.py`

`importlib` でコマ6の `Codegen06` を継承した `Codegen07` に、以下の機能を拡張する。

実装対象	役割
<code>collect_decls_Block(node)</code>	ブロック内の各 <code>stmt</code> を再帰的に収集
<code>collect_decls_If(node)</code>	<code>then</code> 側と <code>else_</code> 側を収集
<code>collect_decls_While(node)</code>	<code>body</code> を収集
<code>collect_decls_For(node)</code>	<code>body</code> を収集
<code>_reset_func_state(node)</code>	全状態の初期化と再帰的宣言収集。 <code>align_to</code> した <code>frame_size</code> を返す
<code>_emit_func_body(body)</code>	<code>body</code> 全体を <code>gen_stmt</code> に渡す形に変更

7 tests/

ファイル	内容	期待値
<code>many_locals.c</code>	6 変数の総和	21
<code>nested_block.c</code>	入れ子ブロック内の変数宣言	7
<code>eight_locals.c</code>	8 変数（アラインメント境界）	36
<code>minimal.c</code>	最小関数（ローカル変数なし）	42

`many_locals.c` | 多数のローカル変数 | 対応する `.ans` を参照 |

`nested_decl.c` | 入れ子ブロック内の宣言 | 対応する `.ans` を参照 |

`early_return.c` | 複数の `return` 文 | 対応する `.ans` を参照 |

8 テスト

```
python3 scaffold/test_runner.py sessions/07_functions_abi
```

確認するポイント:

- `ra` と `s0` を保存・復元する
- フレームサイズを 16 バイト境界に揃える
- 入れ子ブロック内の変数宣言も事前に収集する
- ローカル変数が 0 個でも正しく動く

9 注意

この回では、関数呼び出し、引数、戻り値の受け渡し、再帰はまだ扱わない。それらは第 08 回で扱う。

この回では、C の正確なブロックスコープは実装しない。関数内のすべての変数を関数フレームに一律で確保する。